Guida Pratica Java: Compilazione, JAR, Classpath e Package

---

1. Primo Passo: Compilare ed Eseguire un Singolo File Java

Che cosa servono i comandi principali?

- `javac` (Java Compiler): Traduce il codice che scrivi tu (file `.java`) in un linguaggio capito da Java (file `.class` chiamato **Bytecode**).
- `java` (Java Virtual Machine): Esegue il file `.class`.

Esercizio 1

Step 1: Il Codice

Apri **Notepad** (Blocco Note), incolla questo codice e salvalo con il nome `Saluto.java` nella cartella `C:\JavaTest` (creala se non esiste).

```
public class Saluto {
    public static void main(String[] args) {
        System.out.println("Ciao! Il tuo primo programma Java funziona!");
    }
}
```

Step 2: Riga di Comando (CMD)

1. Apri il Prompt dei Comandi.
2. Vai nella cartella del file scrivendo:

   ```
   cd C:\JavaTest
   ```

3. Compila il file `.java`:

   ```
   javac Saluto.java
   ```

   > **Cosa succede?** Nella cartella compare un nuovo file: `Saluto.class`.

4. Esegui il programma usando **solo il nome della classe** (senza estensione `.class`):

```
java Saluto
```

---

2. Cos'è un File `.JAR`?

Un file `.jar` (Java ARchive) è semplicemente una **cartella zippata** che contiene:

1. I file `.class` compilati.
2. Un file di configurazione speciale chiamato `MANIFEST.MF`.

> **A cosa serve?** Serve a impacchettare tutto il progetto in un unico file facile da distribuire ed eseguire.

---

3. Progetto Multi-File e Creazione di un File `.JAR`

Creiamo un piccolo programma formato da **2 file**.

Esercizio 2

File 1: `Calcolatrice.java`

Salva questo file in `C:\JavaTest`:

```
public class Calcolatrice {
    public int somma(int a, int b) {
        return a + b;
    }
}
```

File 2: `MainApp.java`

Salva anche questo file in `C:\JavaTest`:

```
public class MainApp {
    public static void main(String[] args) {
        Calcolatrice calc = new Calcolatrice();
        int risultato = calc.somma(5, 3);
        System.out.println("La somma di 5 + 3 e': " + risultato);
    }
}
```

Procedura di Compilazione e Impacchettamento

Step 1: Compilare tutti i file

Dal prompt dei comandi in `C:\JavaTest`, compila tutti i file in un colpo solo:

```
javac *.java
```

(Troverai i file `Calcolatrice.class` e `MainApp.class` nella cartella).

Step 2: Creare il file `.jar` ed eseguirlo

Per dire a Java da quale classe deve partire l'esecuzione (la classe che contiene il `main`), usiamo il comando `jar` con l'opzione `e` (entry point):

```
jar cfe MioProgramma.jar MainApp *.class
```

- **`c`**: Crea (Create) un nuovo file JAR.
- **`f`**: Specifica il nome del File JAR (`MioProgramma.jar`).
- **`e`**: Indica il punto di Entry (la classe principale `MainApp`).
- **`*.class`**: Include tutti i file `.class` presenti nella cartella.

Step 3: Eseguire il file `.JAR`

Ora puoi eseguire l'archivio pronto con:

```
java -jar MioProgramma.jar
```

---

4. Concetti Chiave del `.JAR` e Ispezione Contenuto

Come fa Java a sapere quale classe eseguire?

All'interno del file `.jar` c'è un file di testo chiamato `META-INF/MANIFEST.MF`. Contiene una riga come questa:

```
Main-Class: MainApp
```

Come "guardare dentro" un file `.jar`?

Ci sono due modi semplici:

1. **Da Riga di Comando:**

   ```
   jar tf MioProgramma.jar
   ```

(Mostra l'elenco dei file contenuti).

2. **Con un programma di archiviazione (7-Zip / WinRAR):**
    - Puoi rinominare l'estensione da `.jar` a `.zip` oppure aprirlo direttamente con 7-Zip per esplorarlo come una normale cartella.

---

5. Il Classpath: Cos'è e Come si Imposta

Cos'è il Classpath?

Il **Classpath** è la mappa dei percorsi in cui Java va a cercare i file `.class` o le librerie esterne necessarie per eseguire il programma.

Come si imposta?

1. **Opzione temporanea via riga di comando (consigliata):** Usi il parametro `-cp` (oppure `-classpath`):

```
java -cp C:\JavaTest MainApp
```

2. **Includere più percorsi o librerie:** Si separano i percorsi con il punto e virgola `;` (su Windows):

```
java -cp .;C:\Librerie\estensione.jar MainApp
```

(Il punto `.` indica la cartella corrente).

---

6. I Package e la Struttura delle Cartelle

La Regola d'Oro dei Package:

> **Il nome del package deve corrispondere ESATTAMENTE alla struttura delle sottocartelle nel disco.**

Se dichiari `package scuola.matematica;`, il file `.java` (e il suo `.class`) **deve** trovarsi dentro la cartella `scuola\matematica`.

Esercizio 3: Package in Pratica

Step 1: Crea la struttura delle cartelle

Nella tua cartella `C:\JavaTest`, crea una sottocartella chiamata `scuola` e, dentro `scuola`, crea una cartella chiamata `matematica`.

Percorso finale: `C:\JavaTest\scuola\matematica`

Step 2: Crea il file del Package

Incolla questo codice su Notepad e salvalo come `Operazioni.java` dentro `C:\JavaTest\scuola\matematica`:

```
package scuola.matematica;

public class Operazioni {
    public static void mostraMessaggio() {
        System.out.println("Esecuzione da un file dentro il package
scuola.matematica!");
    }
}
```

Step 3: Crea il file principale (Main)

Incolla questo codice su Notepad e salvalo come `TestPackage.java` direttamente nella cartella radice `C:\JavaTest`:

```
import scuola.matematica.Operazioni;

public class TestPackage {
    public static void main(String[] args) {
        Operazioni.mostraMessaggio();
    }
}
```

Compilazione ed Esecuzione con Package

1. Apri il prompt e posizionati nella cartella principale:

   ```
   cd C:\JavaTest
   ```

2. Compila specificando il percorso del file nel package e il main:

   ```
   javac scuola/matematica/Operazioni.java TestPackage.java
   ```

3. Esegui il programma principale:

   ```
   java TestPackage
   ```

Schema Riassuntivo Comandi Utili

Azione	Comando
Compilare un file	`javac NomeFile.java`
Eseguire una classe	`java NomeClasse`
Creare un file .JAR eseguibile	`jar cfe Nome.jar NomeClassePrincipale *.class`
Eseguire un file .JAR	`java -jar Nome.jar`
Vedere il contenuto di un .JAR	`jar tf Nome.jar`
Specifica il Classpath	`java -cp .;\percorso Classe`